

ISA 444/544: Business Forecasting

11: Forecasting Environment and Baseline Forecasts

Fadel M. Megahed, PhD

Raymond E. Glos Professor in Business
Farmer School of Business
Miami University

 @FadelMegahed

 fmegahed

 fmegahed@miamioh.edu

 Automated Scheduler for Office Hours

Fall 2025

Learning Objectives for Today's Class

- Install and import Nixtla's libraries ([StatsForecast](#), [MLForecast](#), [NeuralForecast](#), [UtilsForecast](#), and [TimeGPT](#)) for forecasting
- Distinguish fixed window from rolling-origin
- Introduce forecast accuracy metrics (MAE, MAPE, RMSE)
- Recognize the importance of baseline forecasts in time series forecasting
- Identify common baseline forecasts (Naive, Seasonal Naive, etc.)
- Implement [baseline models with StatsForecast](#)

The Nixtlaverse Open-Source Forecasting Libraries

Nixtla's Forecasting Libraries



Nixtla

Open Source Time Series Ecosystem

2.5k followers United States of America <https://www.nixtla.io/> @nixtlainc company/nixtlainc in/mergenthaler ops@nixtla.io

Overview Repositories 38 Projects Packages People 4 Sponsoring

Pinned

**statsforecast** Public
Lightning ⚡ fast forecasting with statistical and econometric models.
Python 4.5k 331

**neuralforecast** Public
Scalable and user friendly neural 🧠 forecasting algorithms.
Python 3.8k 452

**hierarchicalforecast** Public
Probabilistic Hierarchical forecasting 🏠 with statistical and econometric methods.
Python 700 90

**mlforecast** Public
Scalable machine 🤖 learning for time series forecasting.
Python 1.1k 102

**nixtla** Public
TimeGPT-1: production ready pre-trained Time Series Foundation Model for forecasting and anomaly detection. Generative pretrained transformer for time series trained on over 100B data points. It's ...
Jupyter Notebook 2.9k 239

**nixtlar** Public
R SDK for TimeGPT
R 40 8

People



Top languages

Python Jupyter Notebook
JavaScript CSS R

Most used topics

forecasting time-series machine-learning
python baselines

Repositories

Find a repository... Type Language Sort

utilsforecast Public
Helper functions to plot, evaluate, preprocess and engineer features for forecasting

Nixtla's Forecasting Libraries

Nixtla provides **several open-source Python libraries** (and a closed source **TimeGPT** tool accessible via API calls) for **scalable forecasting tasks**. These libraries are **relatively** easy to use and can be integrated into your future forecasting workflows:

- **StatsForecast** – Fast & scalable statistical models (**ARIMA**, **ETS**, etc.).
- **MLForecast** – Machine learning-based forecasting (e.g., **XGBoost**, **LightGBM**).
- **NeuralForecast** – Deep learning models for time series (e.g., **NBEATS**, **NHITS**, and **TFT**).
- **UtilsForecast** – Utility functions for plotting, evaluation, etc.
- **TimeGPT** – an AI transformer-powered forecasting API that requires minimal tuning.

These libraries enable **forecasting at scale, which you will need in practice**.

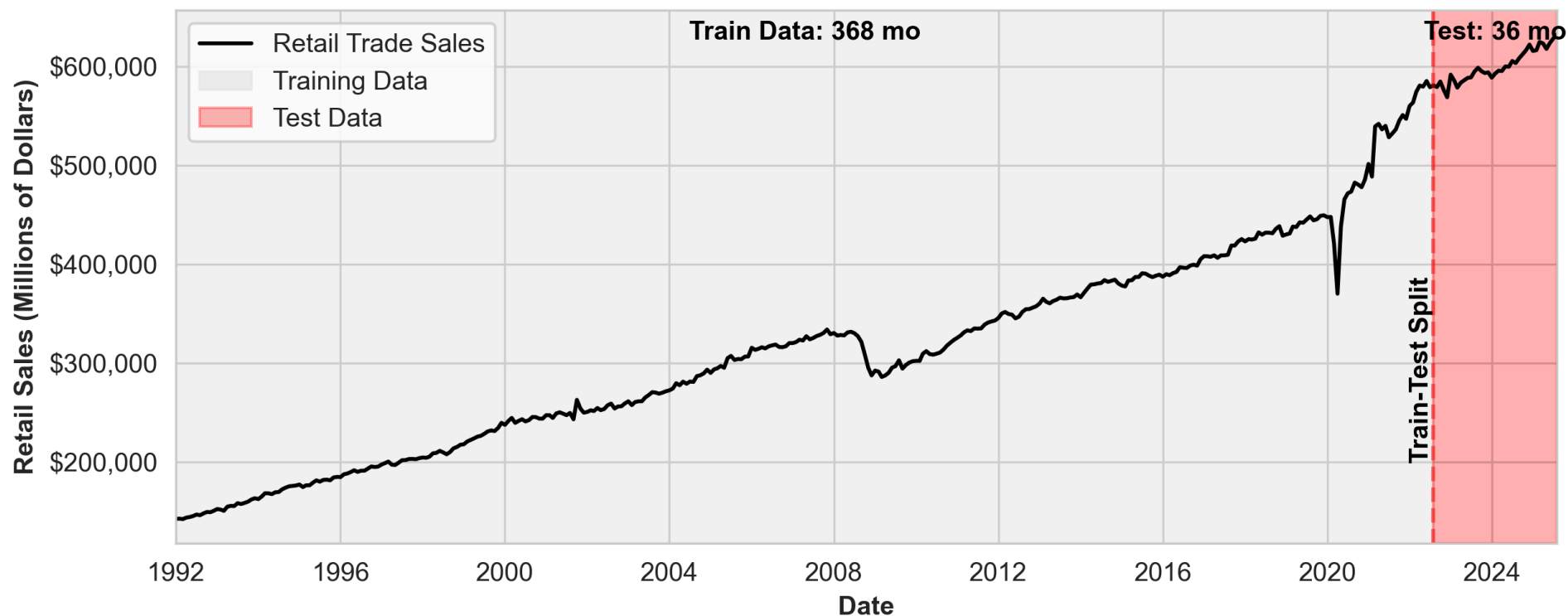
Fixed Window vs. Rolling-Origin

The Fixed Window Evaluation Approach

- **Fixed Window** is the simplest approach to splitting your time series data into a training and a testing/holdout set.
- The **goal** is to **train your model on the training set** and **evaluate its performance on the testing set (the last k observations in the data)**.
 - **Note that this is quite different than traditional machine learning applications for cross sectional data.**
- The evaluation on the **testing/holdout** set can serve two purposes:
 - **Model Evaluation:** Assess the model's **performance on unseen data** (since it is not used during training, and hence acts as a proxy for the model's performance on future data).
 - **Model Selection:** Compare the **performance of different models** to select the **best one**.
 - Note that using this approach for model selection is **reasonable if your models do not involve hyperparameter tuning** (otherwise, you may overfit to the testing set).

The Fixed Window Evaluation Approach

Retail Trade Sales (RSXFS): Train-Test Split
(Test set is often set to a multiple of the period)

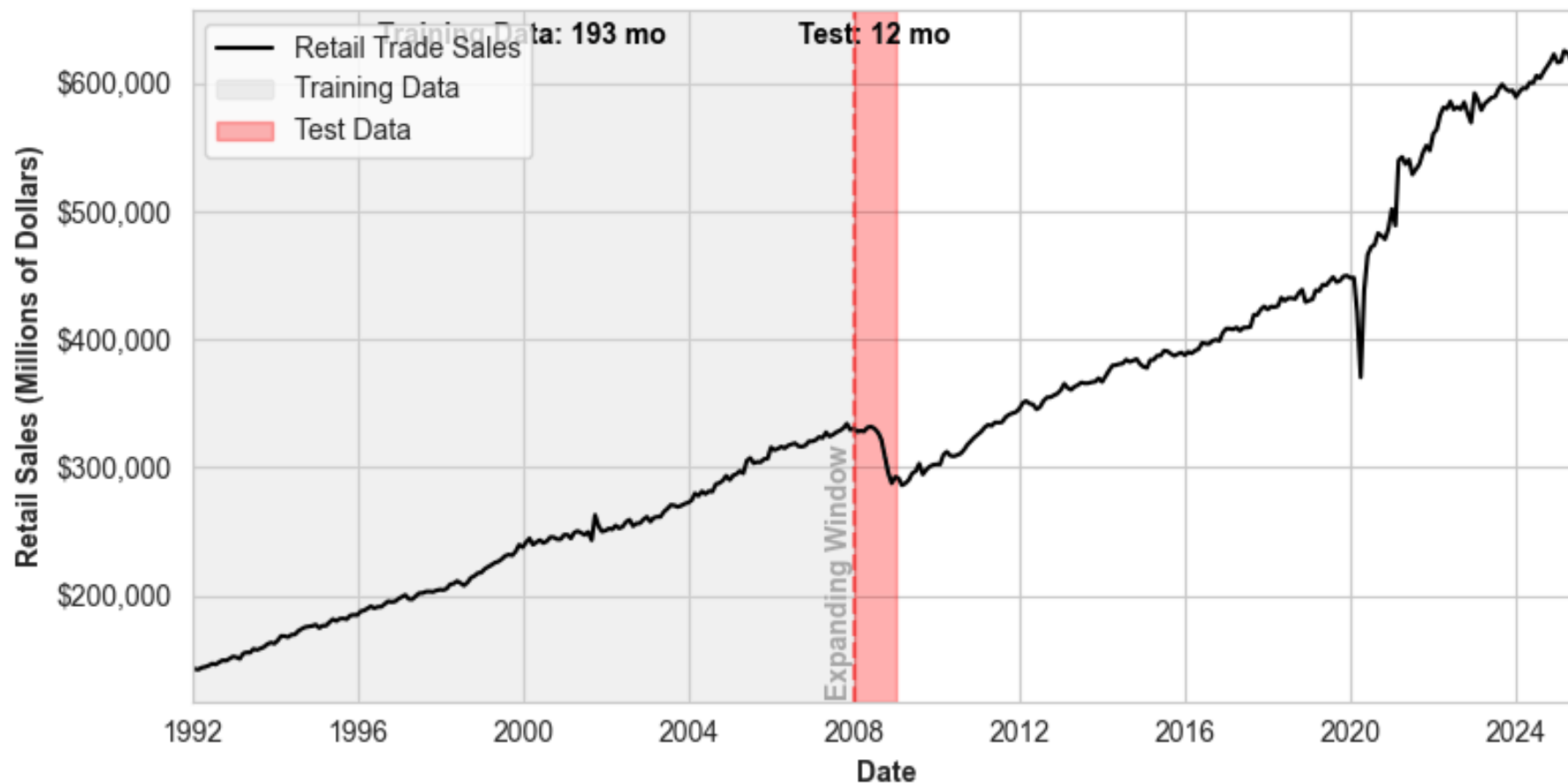


The Rolling-Origin Evaluation Approach

- **Rolling-Origin Evaluation** is a method for splitting time series data into training and testing sets where the testing sets move forward over time.
- The **goal** is to **train the model on a subset of past observations** and evaluate its **performance on a future testing set at multiple time steps**.
- The key difference from a fixed window approach is that the **testing set shifts forward, allowing for multiple evaluations**:
 - The **training set may expand (expanding window)** or **remain fixed (rolling window)**.
 - This ensures that model performance is assessed across **different points in time**.
 - It **reduces sensitivity to the initial split point** and provides a more **robust evaluation** of model performance over time.

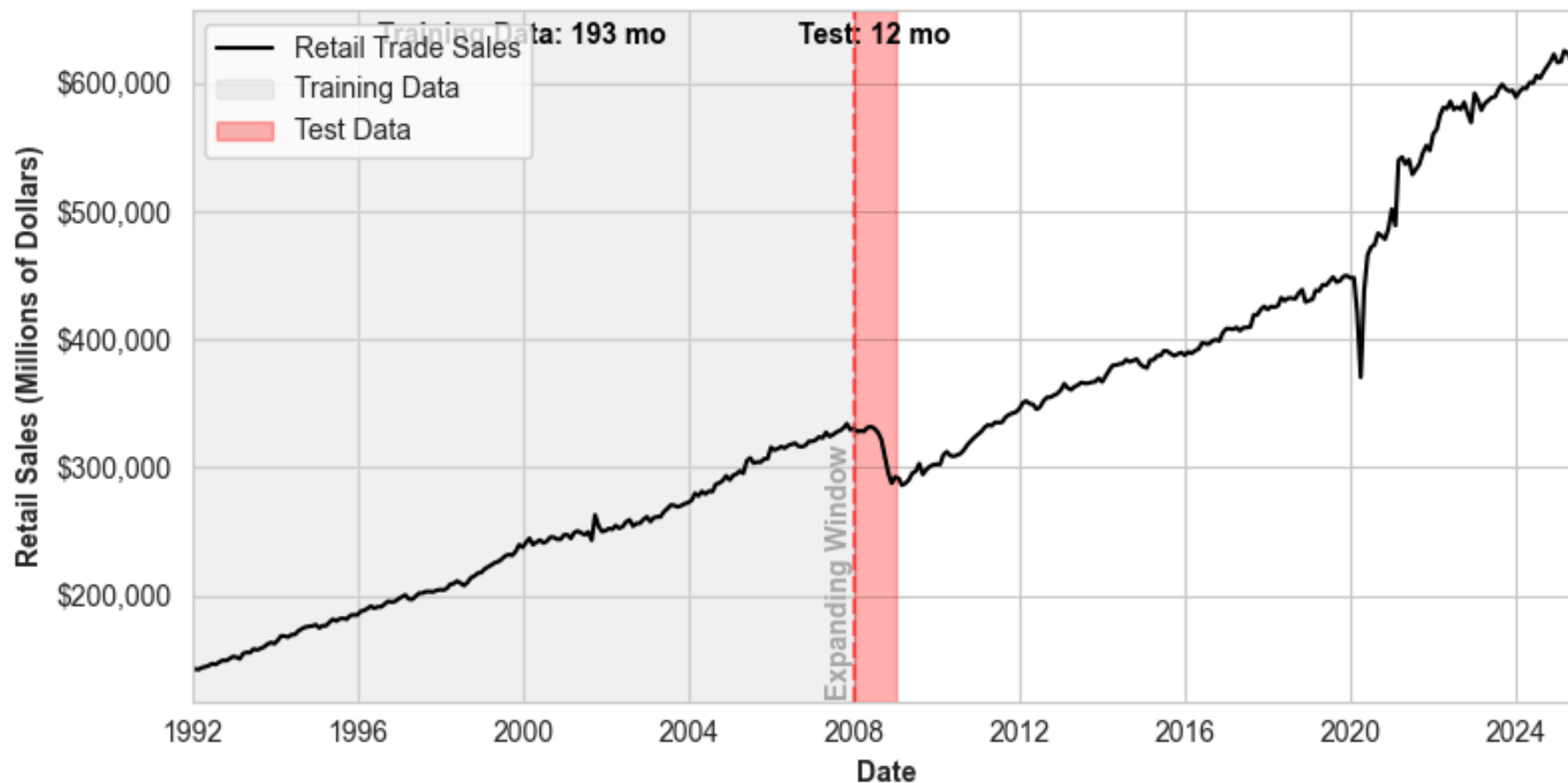
Expanding Window Evaluation (By 1 Month)

Expanding Window Approach for Time Series Forecasting
(Training window expands by 1 month at a time)



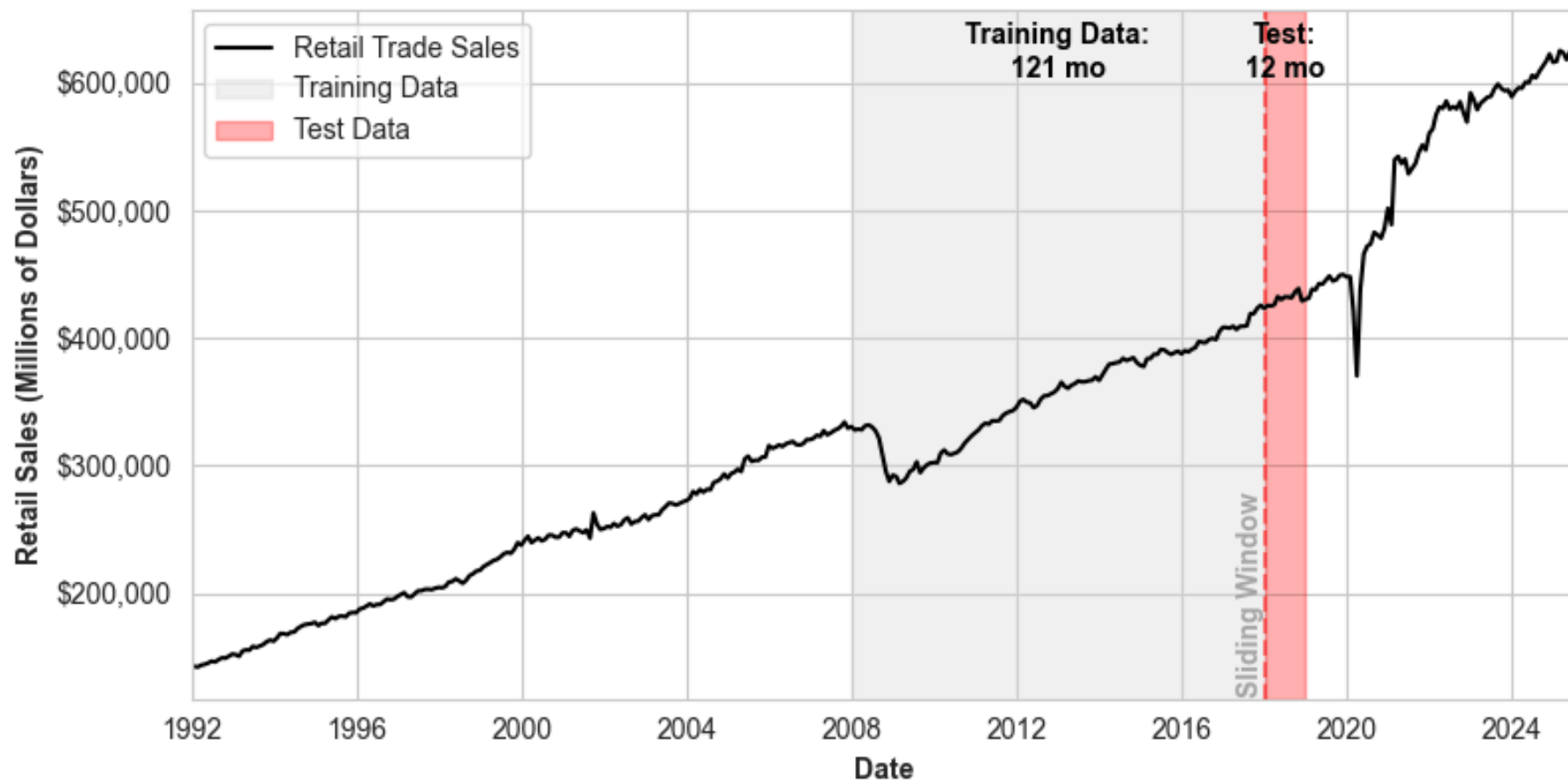
Expanding Window Evaluation (By 12 Month)

Expanding Window Approach for Time Series Forecasting
(Training window expands by 12 months at a time)



Rolling Non-Expanding Window Evaluation

Sliding Window Approach for Time Series Forecasting
(Fixed 10-year training window slides by 12 months)



Cross Validation within the Nixtlaverse

Perform time series cross-validation

Once the `StatsForecast` object has been instantiated, we can use the `cross_validation` method, which takes the following arguments:

- `df` : training data frame with `StatsForecast` format
- `h` (int): represents the h steps into the future that will be forecasted
- `step_size` (int): step size between each window, meaning how often do you want to run the forecasting process.
- `n_windows` (int): number of windows used for cross-validation, meaning the number of forecasting processes in the past you want to evaluate.

For this particular example, we'll use 3 windows of 24 hours.

```
cv_df = sf.cross_validation(  
    df = df,  
    h = 24,  
    step_size = 24,  
    n_windows = 3  
)
```

The `cv_df` object is a new data frame that includes the following columns:

- `unique_id` : series identifier
- `ds` : timestamp or temporal index
- `cutoff` : the last timestamp or temporal index for the `n_windows`.
- `y` : true value
- `"model"` : columns with the model's name and fitted value.

☰ On this page

Introduction

Install libraries

Load and explore the data

Train model

Perform time series cross-validation

Evaluate results

References

Note: The `cross_validation` method can also be applied to other NixtlaForecast objects (e.g., `MLForecast`, `NeuralForecast`) to perform cross-validation for machine learning and deep learning models. See the [StatsForecast Cross Validation Tutorial](#) to access the page shown above.

Activity: The `cross_validation` Method

`cross_validation` method from Nixtla implements:

0

Fixed Window Evaluation

0

Expanding Rolling Window

0

Non-Expanding Rolling Window

Recap of Fixed vs. Rolling-Origin

- **Fixed Window:**
 - **Simplest approach** to splitting data into training and testing sets.
 - **Testing set is fixed** and **does not move forward** over time.
 - **Provides a single evaluation** of model performance.
- **Rolling-Origin:**
 - **Testing set moves forward** over time.
 - **Training set may expand or remain fixed.**
 - **Provides multiple evaluations** of model performance.
 - **Reduces sensitivity to the initial split point.**
 - **More robust evaluation** of model performance over time.

Recap of Fixed vs. Rolling-Origin

- In practice, the **rolling-origin approach is preferred** for time series forecasting tasks since it mimics the real-world scenario of forecasting future data points. The choice of:
 - **Expanding vs. Rolling Window**,
 - **Window Size**, and
 - **Step Size** depends on the specific forecasting task and the data at hand.

Forecast Accuracy Metrics

Model Performance Evaluation in the Nixtlaverse

```
from utilsforecast.losses import *
```

evaluate

```
evaluate (df:~AnyDFTYPE, metrics:List[Callable],  
         models:Optional[List[str]]=None,  
         train_df:Optional[~AnyDFTYPE]=None,  
         level:Optional[List[int]]=None, id_col:str='unique_id',  
         time_col:str='ds', target_col:str='y',  
         agg_fn:Optional[str]=None)
```

Model Performance Evaluation in the Nixtlaverse (Cont.)

	Type	Default	Details
df	AnyDfType		Forecasts to evaluate. Must have <code>id_col</code> , <code>time_col</code> , <code>target_col</code> and models' predictions.
metrics	List		Functions with arguments <code>df</code> , <code>models</code> , <code>id_col</code> , <code>target_col</code> and optionally <code>train_df</code> .
models	Optional	None	Names of the models to evaluate. If <code>None</code> will use every column in the dataframe after removing <code>id</code> , <code>time</code> and <code>target</code> .
train_df	Optional	None	Training set. Used to evaluate metrics such as <code>mase</code> .
level	Optional	None	Prediction interval levels. Used to compute losses that rely on quantiles.
id_col	str	unique_id	Column that identifies each serie.
time_col	str	ds	Column that identifies each timestep, its values can be timestamps or integers.
target_col	str	y	Column that contains the target.
agg_fn	Optional	None	Statistic to compute on the scores by <code>id</code> to reduce them to a single number.
Returns	AnyDfType		Metrics with one row per (id, metric) combination and one column per model. If <code>agg_fn</code> is not <code>None</code>, there is only one row per metric.

Losses

The most important train signal is the forecast error, which is the difference between the observed value y_τ and the prediction $\hat{y}_\tau$, at time y_τ :

$$e_\tau = y_\tau - \hat{y}_\tau \quad \tau \in \{t + 1, \dots, t + H\}$$

The train loss summarizes the forecast errors in different evaluation metrics.

Scale-Dependent Errors: mae

MAE measures the relative prediction accuracy by averaging the absolute deviations between forecasts and actual values.

$$\text{MAE}(y_\tau, \hat{y}_\tau) = \frac{1}{H} \sum_{\tau=t+1}^{t+H} |y_\tau - \hat{y}_\tau|$$

- **Interpretation:** Provides a straightforward measure of forecast accuracy; lower MAE indicates better performance.
- **Characteristic:** Does not penalize larger errors more than smaller ones; treats all errors equally.

Scale-Dependent Errors: rmse

RMSE is the square root of the average of the squared differences between forecasts and actual values.

$$\text{RMSE}(y_\tau, \hat{y}_\tau) = \sqrt{\frac{1}{H} \sum_{\tau=t+1}^{t+H} (y_\tau - \hat{y}_\tau)^2}$$

- **Interpretation:** Emphasizes larger errors due to squaring; useful when large errors are particularly undesirable.
- **Characteristic:** Penalizes large errors more than MAE (i.e., more sensitive to outliers compared to MAE).

Percentage Errors: mape

MAPE calculates the average absolute error as a percentage of actual values.

$$\text{MAPE}(y_\tau, \hat{y}_\tau) = \frac{1}{H} \sum_{\tau=t+1}^{t+H} \left| \frac{y_\tau - \hat{y}_\tau}{y_\tau} \right|$$

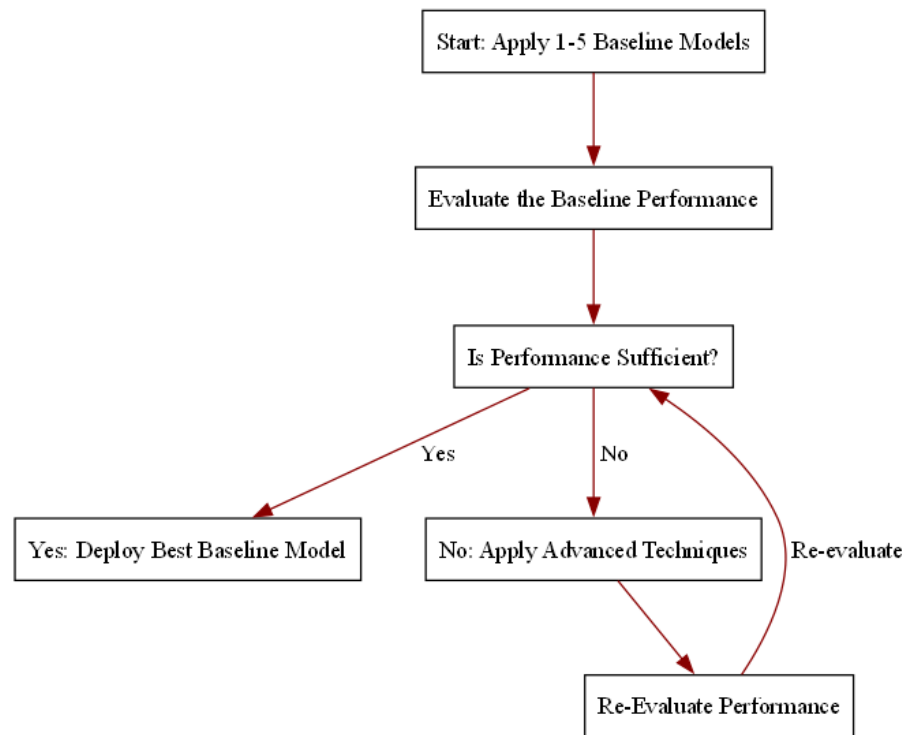
Interpretation: Expresses forecast accuracy as a percentage; lower MAPE indicates better performance.

Characteristic: Can be misleading if actual values are close to zero, leading to extremely high MAPE values.

Importance of Baseline Forecasts

Why do we need Baseline Forecasts?

- Baseline forecasts are **simple, yet powerful**.
- Baseline forecasts serve as **important benchmarks**:
 - They **provide a performance "floor"** for more complex models.
 - Help **assess if advanced models can add meaningful value**.
- They are **easy to interpret** and **require minimal computational resources**.

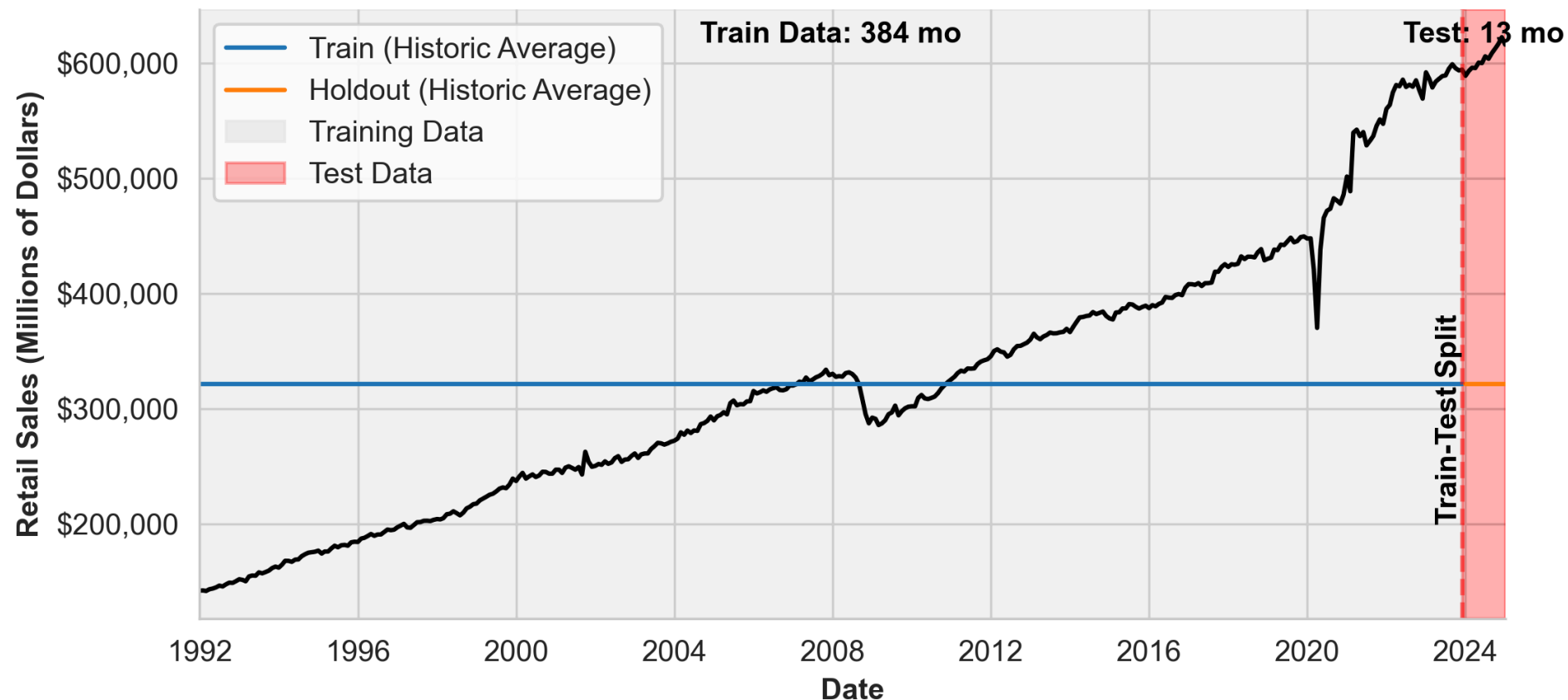


Common Baseline Forecasts

Historic Mean/Average

- **Historic Mean/Average:** The simplest baseline forecast is to use the average of the historical data as the forecast for all future periods.
- **Mathematical Representation:** $\hat{y}_{t+h} = \frac{1}{t} \sum_{i=1}^t y_i$
- **Assumptions about the Data Generating Process (DGP):**
 - Future values will be similar to the past.
 - The data generating process is **relatively stationary (fixed mean with random noise)**, with **no seasonality** and **no trends** (obviously since the mean is fixed).

Historic Mean/Average: Fixed Window



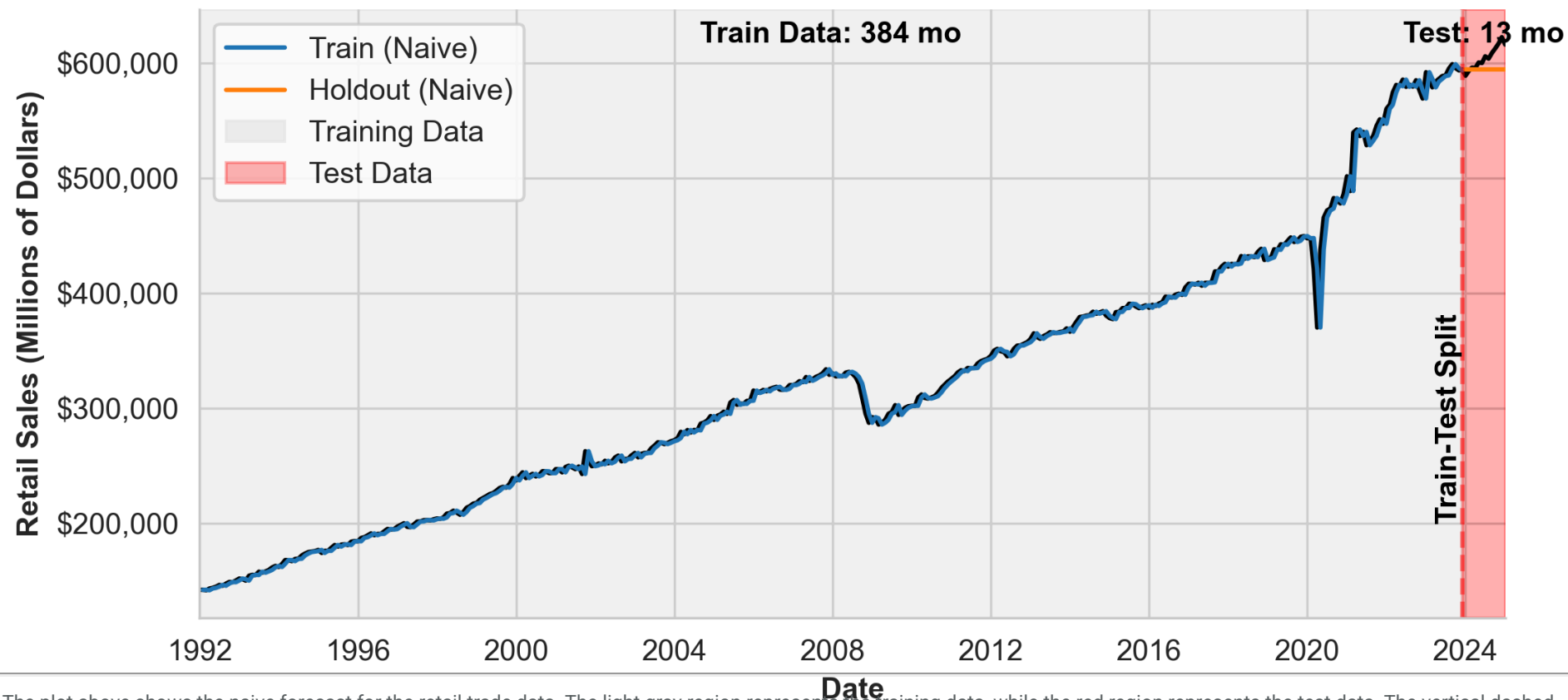
Notes: The plot above shows the historic average forecast for the retail trade data. The light gray region represents the training data, while the red region represents the test data. The vertical dashed line indicates the split point between the training and test data. The training data is used to fit the model, while the test data is used to evaluate the model's performance.

Obviously, the historic average forecast is a straight line, as it is the same for all future periods. Furthermore, it is a terrible choice for this dataset since it clearly has a positive trend.

Naive Forecast

- **Naive Forecast:** The naive forecast assumes that the future value will be the same as the last observed value.
- **Mathematical Representation:** $\hat{y}_{t+h} = y_t$
- **Assumptions about the Data Generating Process (DGP):**
 - Future values will be similar to the most recent past value.
 - The data generating process can either be **relatively stationary** or **follow a random walk**.

Naive Forecast: Fixed Window



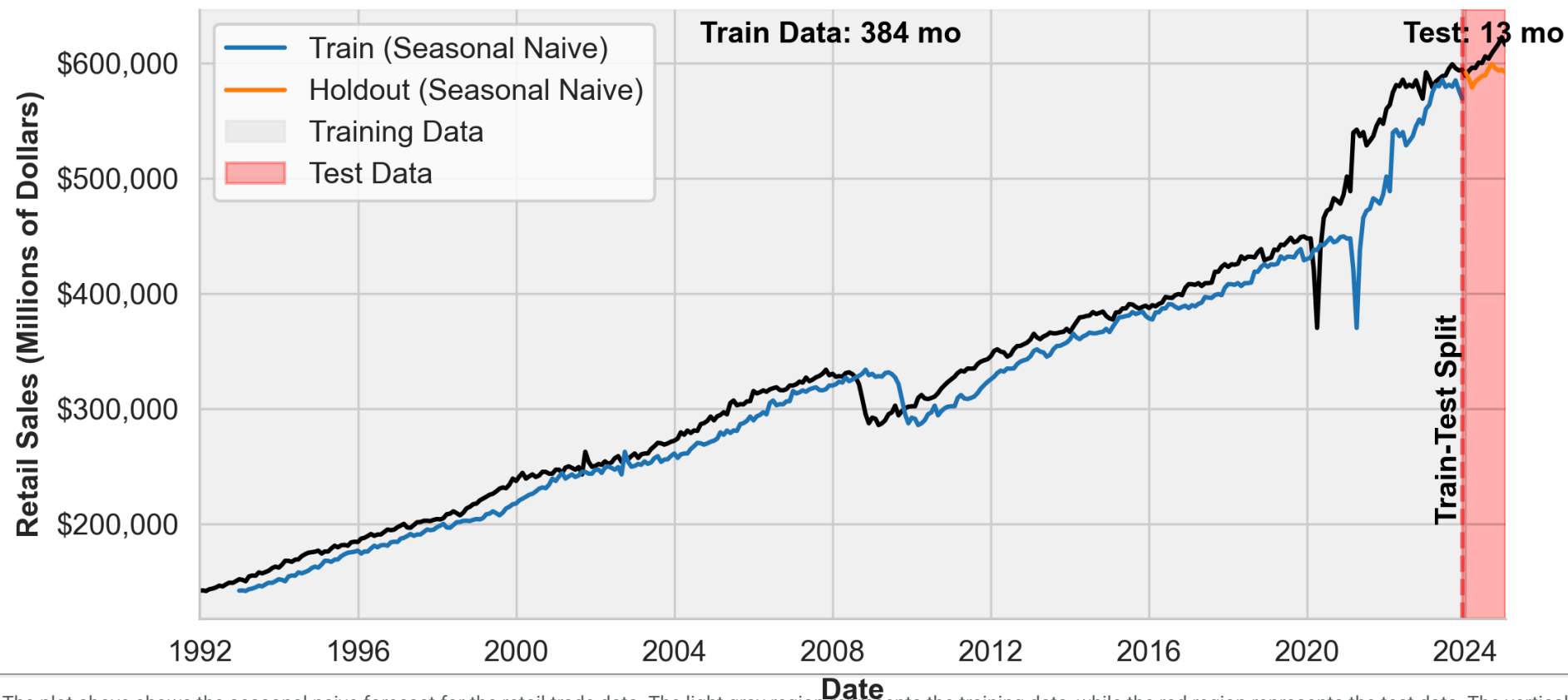
Notes: The plot above shows the naive forecast for the retail trade data. The light gray region represents the training data, while the red region represents the test data. The vertical dashed line indicates the split point between the training and test data. The training data is used to fit the model, while the test data is used to evaluate the model's performance.

The naive forecast lags the training data, and is a straightline for the holdout dataset since we do not update the naive forecast for the fixed window approach. Similar to the historic average forecast, the naive forecast is a terrible choice for this dataset since it clearly has a positive trend.

Seasonal Naive Forecast

- **Seasonal Naive Forecast:** The seasonal naive forecast assumes that the future value will be the same as the last observed value from the same season.
- **Mathematical Representation:** $\hat{y}_{t+h} = y_{t+h-m}$, where m is the seasonal period.
- **Assumptions about the Data Generating Process (DGP):**
 - Future values will be similar to the most recent past value from the same season.
 - The data generating process will be **seasonal but with no trends**.

Seasonal Naive Forecast: Fixed Window



Notes: The plot above shows the seasonal naive forecast for the retail trade data. The light gray region represents the training data, while the red region represents the test data. The vertical dashed line indicates the split point between the training and test data. The training data is used to fit the model, while the test data is used to evaluate the model's performance.

The seasonal naive forecast lags the training data, and is worse than the naive model (since now the data lags by 12 months instead of 1 month); it would be better if the data is seasonal with no trends.

Implementing Baseline Models with StatsForecast

Live Demo

We will now implement the historic average, naive, and seasonal naive forecasts using the [StatsForecast](#) library. Our in-class implementation will highlight both the fixed window and rolling-origin approaches.

Recap

Summary of Main Points

By now, you should be able to do the following:

- Install and import Nixtla's libraries ([StatsForecast](#), [MLForecast](#), [NeuralForecast](#), [UtilsForecast](#), and [TimeGPT](#)) for forecasting
- Distinguish fixed window from rolling-origin
- Introduce forecast accuracy metrics (MAE, MAPE, RMSE)
- Recognize the importance of baseline forecasts in time series forecasting
- Identify common baseline forecasts (Naive, Seasonal Naive, etc.)
- Implement [baseline models with StatsForecast](#)



Review and Clarification



- **Class Notes:** Take some time to revisit your class notes for key insights and concepts.
- **Zoom Recording:** The recording of today's class will be made available on Canvas approximately 3-4 hours after the session ends.
- **Questions:** Please don't hesitate to ask for clarification on any topics discussed in class. It's crucial not to let questions accumulate.